Dominando Testes Unitários

Uma Jornada Prática com Pytest para Desenvolvimento de Software Robusto

Sumário

- Introdução e Qualidade de Software
- Conceitos e Benefícios de Testes Unitários
- Pytest: Escolha e Funcionalidades
- Configuração e Estrutura de Testes
- Fixtures e Parametrização Avançada

Dominando Testes Unitários

Nesta aula, exploraremos os *Testes Unitários* (testes que verificam pequenas partes isoladas do código) com *pytest*, uma ferramenta robusta para garantir a *qualidade de software* (o grau em que um software atende aos requisitos e expectativas do usuário). Prepare-se para elevar suas habilidades de desenvolvimento e construir aplicações mais confiáveis e eficientes!

Qualidade de Software: A Base

Software de qualidade vai além da funcionalidade; ele é *robusto* (resiliente a falhas), *confiável* (opera consistentemente) e *manutenível* (fácil de modificar). Isso garante a estabilidade e longevidade de qualquer projeto técnico, tornando os testes unitários essenciais para validar cada componente.

Laboratório de testes de materiais de borracha com equipamentos especializados.

O que são Testes Unitários?

São **verificações automatizadas** de pequenas partes do código, como *funções* ou *métodos*. Garantem que cada unidade isolada funcione corretamente, focando na **granularidade** para detectar falhas específicas.

Por Que Testar Unidades?

- Identifica falhas no código *rapidamente*, reduzindo custos de correção.
- Garante que novas mudanças não quebrem funcionalidades existentes.
- Aumenta a *confiabilidade* do software e a *qualidade* do produto.
- Serve como *documentação viva*, facilitando a compreensão do código.

Ecossistema de Testes

Pirâmide de Testes

Testes Unitários formam a *base*, sendo os mais numerosos, rápidos e de baixo custo de manutenção.

Testes de Integração

Verificam a comunicação entre módulos ou componentes, por exemplo, a integração com um banco de dados.

Testes E2E

Simulam o fluxo completo do usuário (end-to-end), avaliando o sistema como um todo, do front-end ao back-end.

Frameworks de Teste no Python

unittest (Padrão)

O `unittest` é o módulo de teste padrão do Python. Ele segue o estilo xUnit (um framework de teste para diversas linguagens), exigindo classes e métodos específicos. É robusto para testes unitários, mas pode ser mais verboso.

pytest (Popular)

O `pytest` é um framework de teste amplamente adotado, conhecido por sua simplicidade. Ele permite escrever testes de forma mais concisa, usando funções. Suporta *fixtures* (funções que preparam o ambiente para testes) e plugins avançados recursos de plugins.

Por Que Escolher pytest?

- Sintaxe clara e concisa para testes.
- Curva de aprendizado rápida e fácil.
- Poderoso para projetos complexos.
- Plugins (extensões) ampliam funcionalidades.

Funcionalidades Chave do pytest

- Testes com funções Python *simples*.
- Relatórios *detalhados* de execução.
- Fixtures para *setup/teardown* eficiente.
- Parametrização de testes *flexível*.

Configuração Inicial do pytest

1

Instalar pytest via pip

Use `pip install pytest` no terminal para adicionar o *framework* (estrutura de software) ao seu ambiente.

2

Criar Estrutura de Projeto

Organize seus testes em um diretório `tests/` para melhor modularidade (divisão em partes).

3

Primeiro Teste Simples

Crie um arquivo `test_exemplo.py` e adicione uma função de teste `def test_soma():`.

Arquivo de Código (Módulo)

Este é o arquivo Python que contém as funções ou métodos a serem testados. Ele representa a unidade de código que você deseja validar. Por exemplo, uma função que realiza um cálculo específico.

Arquivo de Teste (`test_*.py`)

Este arquivo, com prefixo `test_`, contém as funções de teste. Cada função de teste (`test_nome_da_funcao`) invoca a unidade de código e verifica seu comportamento. O `pytest` automaticamente descobre e executa esses testes.

Convenções de Nomenclatura

- Arquivos de teste: prefixo `test_` (ex: `test_modulo.py`).
- Funções de teste: prefixo `test_` (ex: `test_func_soma()`).
- Facilita a *descoberta automática* (identificação dos testes).
- Essencial para a *automação* (execução eficiente dos testes).

O Poder do `assert`

A instrução `assert` é a base da validação em testes unitários. Ela compara um *valor obtido* com um *valor esperado*, essencial para verificar o comportamento do código. Se a condição for falsa, um `AssertionError` (uma exceção, ou seja, um evento que interrompe o fluxo normal do programa) é levantado, indicando uma falha no teste.

Executando seus Primeiros Testes

1

Executar o Pytest

No terminal, digite `pytest` para rodar todos os testes do diretório atual.

2

Analisar o Resultado

Verde indica teste *passou*, vermelho indica *falhou*. O status é exibido claramente.

3

Automação e Feedback

O `pytest` automatiza a verificação, fornecendo feedback rápido sobre a qualidade do código.

Cenários Diversos com Parametrização

A parametrização do pytest executa o mesmo teste com múltiplas entradas e saídas esperadas. Isso evita a repetição de código, otimizando a cobertura de cenários de teste de forma eficiente.

Aplicando `@pytest.mark.parametrize`

O decorador `@pytest.mark.parametrize` permite executar o mesmo teste com diferentes conjuntos de dados. Define-se os parâmetros e seus valores (tuplas ou lista de tuplas), que são passados como argumentos para a função de teste. Ideal para testar funções como somas com múltiplos inputs.

Fixtures: Preparando o Ambiente

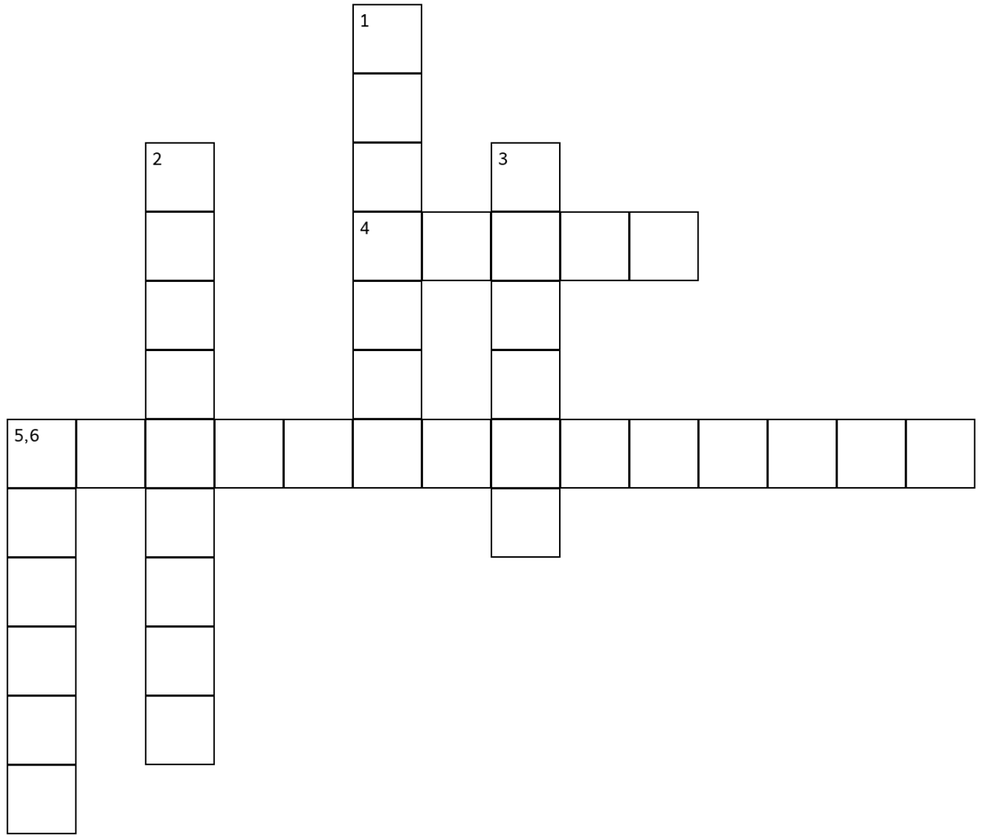
Fixtures são funções que preparam o ambiente de teste (dados, objetos) e realizam a limpeza (*teardown*), garantindo isolamento. Elas asseguram que cada teste comece em um estado conhecido, otimizando a repetibilidade e confiabilidade.

Técnico inspeciona o Landsat 4, preparando o ambiente para testes.

Criando e Usando Fixtures

Declare *fixtures* (funções que fornecem contexto ou dados para testes) com `@pytest.fixture`. Elas configuram ambientes, retornam objetos ou dados, e são injetadas diretamente nos testes como argumentos, simplificando a escrita e manutenção.

CRUZADINHA



HORIZONTAL

4. Verificação automatizada de uma parte específica do código para garantir que funcione conforme o esperado.

5. Técnica que permite executar o mesmo teste várias vezes com diferentes conjuntos de dados de entrada.

VERTICAL

1. Função ou pedaço de código que fornece um ambiente ou dados pré-configurados para um teste, garantindo que ele seja executado de forma isolada e repetível.

2. Métrica que mede a porcentagem de linhas de código, ramos ou declarações que foram executadas por um conjunto de testes.

3. Palavra-chave usada para verificar se uma condição é verdadeira, falhando o teste se a condição for falsa.

6. Estrutura de testes popular em Python para escrita de testes de unidade e integração.

					1 F								
					I								
		2 C			X		3 A						
		O			4 T	E	S	T	E				
		B			U		S						
		E			R		E						
5,6 P	A	R	A	M	E	T	R	I	Z	A	C	A	O
Y		T					T						
T		U											
E		R											
S		A											
T													

Conclusão

- Testes unitários asseguram a qualidade do software, identificando erros precocemente e prevenindo regressões em sistemas complexos.

- O pytest se destaca pela simplicidade na escrita, robustez para projetos técnicos e relatórios detalhados.

- Fixtures e parametrização, `@pytest.fixture` e `@pytest.mark.parametrize` respectivamente, otimizam a cobertura de cenários de teste.

- `assert` é crucial para validar resultados, enquanto convenções de nomeclatura (`test_`) garantem descoberta de testes automáticos.